Университет ИТМО

Отчёт по домашнему заданию № 3

Технология MapReduce

Дисциплина «Инженерия данных»

Вариант № 2

Программист, разработчик на Python

Выполнила:
Бондаренко Анна Андреевна

1. RustFS и исходные данные

Использован кластер RustFS из предыдущего ДЗ: четыре контейнера `rustfs-lab-node1`–`rustfs-lab-node4`. S3 API доступен на порту `19000`, консоль — на `19001`. Конфигурация сохранена в `compose.yaml`.

```
(base) annabondarenko@MacBook-Air-Anna-4 ~ % docker start rustfs-lab-node1 rustfs-lab-node2 rustfs-lab-node3 rustfs-lab-node4
rustfs-lab-node1
rustfs-lab-node2
rustfs-lab-node3
rustfs-lab-node4
(base) annabondarenko@MacBook-Air-Anna-4 ~ % docker ps -a --filter name=rustfs-lab-node --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
NAMES              STATUS              PORTS
rustfs-lab-node3   Up About a minute   9000-9001/tcp
rustfs-lab-node4   Up About a minute   9000-9001/tcp
rustfs-lab-node2   Up About a minute   9000-9001/tcp
rustfs-lab-node1   Up About a minute   127.0.0.1:19000->9000/tcp, 127.0.0.1:19001->9001/tcp
```

Рисунок 1: Запуск четырёх контейнеров RustFS

В бакет `hh-lake` загружен предоставленный файл `it_vacancies_2024.csv`. Размер — 847 619 384 байта, или 808,35 MiB. Путь объекта: `s3://hh-lake/raw/it_vacancies_2024.csv`. После загрузки объект полностью прочитан из S3. Размер и SHA-256 совпали с исходником.

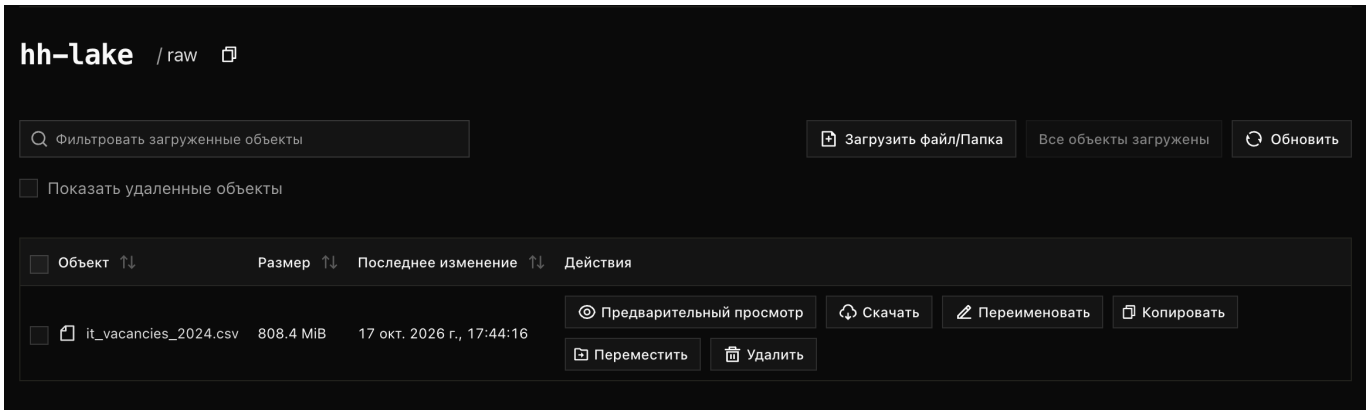


Рисунок 2: Исходный CSV в бакете hh-lake

Из 229 388 записей выбраны вакансии с ролью «Программист, разработчик» и отдельным словом Python в названии без учёта регистра. Python только в списке навыков недостаточно для отбора.

Показатель	Значение
Вакансий варианта № 2	3 892
С указанными навыками	2 984
Без указанных навыков	908
С любой указанной зарплатой	1 446
С рублёвой зарплатой на руки	912

2. Подсчёт MapReduce

```
python scripts/run_mapreduce.py ––upload
```

CSV читается непосредственно из S3 чанками по 1 000 записей. Работают два процесса-маппера, всего обработано 230 чанков. Навыки приводятся к нижнему регистру, пробелы по краям и повторы внутри вакансии удаляются. Мапперы считают частоты сочетаний, родительский процесс суммирует их по ключам.

Вакансия с n уникальными навыками даёт n отдельных навыков, $\binom{n}{2}$ пар и $\binom{n}{3}$ троек. Например, 10 навыков дают 45 пар и 120 троек. Поэтому число сочетаний растёт быстрее числа навыков.

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/run_mapreduce.py --upload
mapreduce: s3://hh-lake/raw/it_vacancies_2024.csv
строк в чанке: 1000, процессов: 2
k=1: сочетаний 1269, вхождений 19727
k=2: сочетаний 17798, вхождений 81302
k=3: сочетаний 115106, вхождений 291454
s3: k=1, порций: 1
s3: k=2, порций: 3
s3: k=3, порций: 15
строк: 229388, вакансий: 3892, чанков: 230
время: 23.295 с
результаты: results/mapreduce
s3: результаты и манифест сохранены в hh-lake/results/mapreduce
```

Рисунок 3: Запуск MapReduce с чтением S3

Результаты сохранены в `hh-lake/results/mapreduce`. Для размеров сочетаний 1, 2 и 3 манифест содержит соответственно 1, 3 и 15 gzip-порций. Их порядок и контрольные суммы указаны в `manifest.json`. Порции ограничены 60 KiB из-за ошибки записи крупных объектов в используемой сборке RustFS на Docker Desktop.

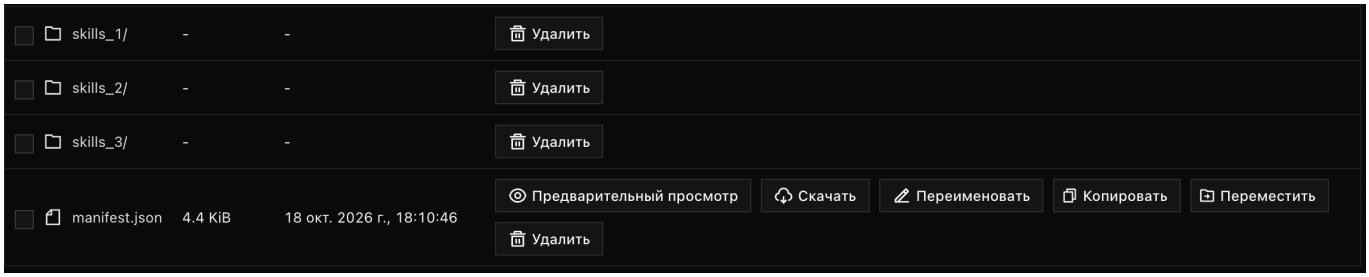


Рисунок 4: Результаты для навыков, пар и троек в RustFS

3. DuckDB и сверка

```
python scripts/run_duckdb.py
python scripts/compare_results.py
```

DuckDB независимо читает тот же объект S3. Условие варианта в `sql/prepare.sql`:

```
where trim(role) = 'Программист, разработчик'
  and regexp_matches(
      lower(name),
      '(^|[^\p{L}\p{N}_])python([^\p{L}\p{N}_]|$)'
  )
```

Для пар и троек навыки соединяются по `vacancy_id` с условиями `a.skill < b.skill` и `b.skill < c.skill`, чтобы исключить перестановки и повторы.

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/run_duckdb.py
duckdb: s3://hh-lake/raw/it_vacancies_2024.csv
k=1: сочетаний 1269, вхождений 19727
k=2: сочетаний 17798, вхождений 81302
k=3: сочетаний 115106, вхождений 291454
вакансий: 3892
время: 15.866 с
результаты: results/duckdb
```

Рисунок 5: Независимый расчёт DuckDB

k	Ключей MR	Ключей SQL	Вхождений MR	Вхождений SQL
1	1 269	1 269	19 727	19 727
2	17 798	17 798	81 302	81 302
3	115 106	115 106	291 454	291 454

Вхождения — сумма частот всех сочетаний соответствующего размера. Проверены все ключи, частоты и ID выбранных вакансий. Расхождений нет.

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/compare_results.py
сверка: mapreduce и duckdb
k=1: ключей 1269/1269, расхождений 0
k=2: ключей 17798/17798, расхождений 0
k=3: ключей 115106/115106, расхождений 0
вакансии совпадают: да
результаты совпадают: да
```

Рисунок 6: Полная сверка MapReduce и DuckDB

Повтор маппера

```
python scripts/run_mapreduce.py --inject-failure --output mapper_failure
python scripts/compare_results.py --left mapreduce --right mapper_failure
```

На чанке 0 смоделирована ошибка задачи до передачи результата редьюсеру. Чанк повторён один раз. Результат совпал с обычным запуском, двойного учёта нет.

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/run_mapreduce.py --inject-failure --output mapper_failure
mapreduce: s3://hh-lake/raw/it_vacancies_2024.csv
строк в чанке: 1000, процессов: 2
ошибка маппера: чанк 0, повтор 1
k=1: сочетаний 1269, вхождений 19727
k=2: сочетаний 17798, вхождений 81302
k=3: сочетаний 115106, вхождений 291454
строк: 229388, вакансий: 3892, чанков: 230
время: 19.34 с
результаты: results/mapper_failure
```

Рисунок 7: Ошибка маппера и повтор чанка

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/compare_results.py --left mapreduce --right mapper_failure
сверка: mapreduce и mapper_failure
k=1: ключей 1269/1269, расхождений 0
k=2: ключей 17798/17798, расхождений 0
k=3: ключей 115106/115106, расхождений 0
вакансии совпадают: да
результаты совпадают: да
```

Рисунок 8: Сверка после повтора

4. Время и память

```
python scripts/benchmark.py --runs 3
```

Каждый подход запущен трижды в отдельном процессе. Измерено время запуска Python, чтения S3, расчёта и записи локальных результатов. Загрузка в S3 и внешняя сверка в замер не входят. Кеши не сбрасывались, `httpfs` установлен заранее.

Память — максимальная наблюдавшаяся сумма RSS родителя и дочерних процессов с паузой опроса 50 мс. Общие страницы могут учитываться повторно. Память RustFS и Docker VM не измеряется.

Попытка	MR, с	MR, MiB	DuckDB, с	DuckDB, MiB
1	21,154	364,89	18,839	1 061,55
2	19,545	369,41	15,814	1 246,50
3	20,616	338,70	14,870	1 309,12

Подход	Медиана времени, с	Максимум RSS, MiB
MapReduce	20,616	369,41
DuckDB	15,814	1 309,12

Во всех шести попытках результаты совпали. Медианное время DuckDB меньше на 23,3%, максимальная сумма RSS больше в 3,54 раза. DuckDB использует буферы SQL-движка и группировок. MapReduce разбирает CSV и суммирует частоты в Python, а также передаёт чанки между процессами. В этих запусках выигрыш по времени сопровождается большим расходом памяти.

Одиночный запуск MapReduce с `--upload` включает загрузку результатов, поэтому его время отличается от таблицы профилирования.

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/benchmark.py --runs 3
mapreduce: запуск 1/3
время: 21.154 с, память: 364.89 mib, сверка: совпало
duckdb: запуск 1/3
время: 18.839 с, память: 1061.55 mib, сверка: совпало
mapreduce: запуск 2/3
время: 19.545 с, память: 369.41 mib, сверка: совпало
duckdb: запуск 2/3
время: 15.814 с, память: 1246.50 mib, сверка: совпало
mapreduce: запуск 3/3
время: 20.616 с, память: 338.70 mib, сверка: совпало
duckdb: запуск 3/3
время: 14.870 с, память: 1309.12 mib, сверка: совпало
метод       медиана, с    максимум rss, mib
mapreduce      20.616              369.41
duckdb         15.814             1309.12
логи: evidence/benchmark_main
```

Рисунок 9: Три запуска каждого подхода с измерением времени и памяти

5. Навыки и зарплаты

Анализ выполнен на той же выборке из 3 892 Python-вакансий. Использованы 912 зарплат в рублях с `gross=false`. Для вилки берётся середина, при одной границе — указанная сумма. Валюты и зарплаты до вычета налогов с этой выборкой не смешиваются.

Для пары база — более частый навык, для тройки — более частая входящая пара. Частота определяется по всей выборке, медианы — по зарплатной части. База включает целевую группу. В целевой группе не менее 10 зарплат.

Разница рассчитывается как $(M_{\text{сочетания}}/M_{\text{базы}} - 1) \cdot 100\%$.

5.1. Пары навыков

```
python scripts/analysis.py salary ––size 2
```

Сочетание	База	Медиана базы, руб.	Медиана группы, руб.	n	Изм., %
bottle + python	python	150 000	375 000	11	+150,0
python + zeromq	python	150 000	375 000	11	+150,0
asyncio + redis	redis	175 000	400 000	19	+128,6
git + html	git	140 000	57 500	20	-58,9
css + git	git	140 000	61 250	14	-56,2
java + python	python	150 000	67 500	12	-55,0

n — число зарплат в целевой группе. В базах Python, Redis и Git соответственно 709, 149 и 275 зарплат.

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/analysis.py salary --size 2
зарплаты: вакансий 3892, рублёвых зарплат на руки 912
размер сочетания: 2, минимум зарплат: 10
рост медианы
  bottle + python
  база python: 150000 → 375000 руб, +150.0%, n=11
  python + zeromq
  база python: 150000 → 375000 руб, +150.0%, n=11
  asyncio + redis
  база redis: 175000 → 400000 руб, +128.6%, n=19
снижение медианы
  git + html
  база git: 140000 → 57500 руб, -58.9%, n=20
  css + git
  база git: 140000 → 61250 руб, -56.2%, n=14
  java + python
  база python: 150000 → 67500 руб, -55.0%, n=12
результаты: results/analysis/salary_pairs.csv
```

Рисунок 10: Пары навыков с положительной и отрицательной разницей медиан

5. Навыки и зарплаты

5.2. Тройки навыков

```
python scripts/analysis.py salary ––size 3
```

Сочетание	База	Медиана базы, руб.	Медиана группы, руб.	n	Изм., %
elasticsearch + python + redis	python + redis	180 000	420 000	11	+133,3
kafka + python + redis	python + redis	180 000	420 000	11	+133,3
asyncio + postgresql + redis	postgresql + redis	180 000	400 000	18	+122,2
git + html + python	git + python	135 000	57 500	20	-57,4
css + git + python	git + python	135 000	61 250	14	-54,6
css + python + sql	python + sql	150 000	70 000	15	-53,3

n — число зарплат в целевой группе. В базах Python + Redis, PostgreSQL + Redis, Git + Python и Python + SQL соответственно 138, 104, 260 и 222 зарплаты.

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/analysis.py salary --size 3
зарплаты: вакансий 3892, рублёвых зарплат на руки 912
размер сочетания: 3, минимум зарплат: 10
рост медианы
  elasticsearch + python + redis
  база python + redis: 180000 → 420000 руб, +133.3%, n=11
  kafka + python + redis
  база python + redis: 180000 → 420000 руб, +133.3%, n=11
  asyncio + postgresql + redis
  база postgresql + redis: 180000 → 400000 руб, +122.2%, n=18
снижение медианы
  git + html + python
  база git + python: 135000 → 57500 руб, -57.4%, n=20
  css + git + python
  база git + python: 135000 → 61250 руб, -54.6%, n=14
  css + python + sql
  база python + sql: 150000 → 70000 руб, -53.3%, n=15
результаты: results/analysis/salary_triples.csv
```

Рисунок 11: Тройки навыков с положительной и отрицательной разницей медиан

Положительная разница связана с добавлением Bottle, ZeroMQ и asyncio. Для базы Python + Redis выделяются Elasticsearch и Kafka. Отрицательная разница наблюдается у групп с HTML, CSS и Java, а для троек — также при добавлении CSS к Python + SQL.

На результат влияет состав вакансий. В группе Bottle + Python все 11 вакансий с зарплатой требуют от трёх лет опыта. В группе Git + HTML 16 из 20 рассчитаны на опыт до трёх лет или без опыта. Поэтому разницу медиан нельзя приписать только навыку. Малые группы чувствительны к отдельным предложениям. Проверка состава групп приведена в `sql/experience.sql`.

6. Сравнение регионов

```
python scripts/analysis.py regions
```

Выбраны три региона с наибольшим числом вакансий варианта № 2. Общие показатели рассчитаны по всем вакансиям региона, включая записи без навыков. Зарплатные показатели используют те же рублёвые суммы на руки.

Регион	Вакансий	С навыками	Зарплат	Доля, %	Медиана, руб.
Москва	1 908	1 430	381	19,97	200 000
Санкт-Петербург	525	403	144	27,43	150 000
Екатеринбург	152	122	45	29,61	150 000

Доля — часть всех вакансий региона с подходящей зарплатой. Для каждого размера ниже приведено самое частое сочетание. Частота считается по всем вакансиям региона, медиана — по записям с зарплатой.

Регион	k	Сочетание	Частота	Зарплат	Медиана, руб.
Москва	1	python	1359	302	200 000
Москва	2	postgresql + python	613	139	200 000
Москва	3	docker + postgresql + python	227	52	225 000
Санкт-Петербург	1	python	391	111	140 000
Санкт-Петербург	2	postgresql + python	190	55	170 000
Санкт-Петербург	3	docker + postgresql + python	69	17	135 000
Екатеринбург	1	python	114	31	150 000
Екатеринбург	2	postgresql + python	55	15	150 000
Екатеринбург	3	git + postgresql + python	22	5	108 000

```
(.venv) annabondarenko@MacBook-Air-Anna-4 mapreduce % python scripts/analysis.py regions
регион                  вакансий  зарплат  медиана, руб
Москва                      1908      381       200000
Санкт-Петербург              525      144       150000
Екатеринбург                 152       45       150000
самые частые сочетания
Москва
  k=1: python, частота 1359, зарплат 302, медиана 200000 руб
  k=2: postgresql + python, частота 613, зарплат 139, медиана 200000 руб
  k=3: docker + postgresql + python, частота 227, зарплат 52, медиана 225000 руб
Санкт-Петербург
  k=1: python, частота 391, зарплат 111, медиана 140000 руб
  k=2: postgresql + python, частота 190, зарплат 55, медиана 170000 руб
  k=3: docker + postgresql + python, частота 69, зарплат 17, медиана 135000 руб
Екатеринбург
  k=1: python, частота 114, зарплат 31, медиана 150000 руб
  k=2: postgresql + python, частота 55, зарплат 15, медиана 150000 руб
  k=3: git + postgresql + python, частота 22, зарплат 5, медиана 108000 руб
результаты: results/analysis/regions.csv, results/analysis/region_skills.csv
```

Рисунок 12: Региональные показатели и частые сочетания

Москва даёт наибольшую выборку и медиану 200 000 руб. В Санкт-Петербурге и Екатеринбурге медиана равна 150 000 руб., на 25% ниже московской. Доля вакансий с подходящей зарплатой в Москве ниже: 19,97% против 27,43% и 29,61%.

Во всех трёх регионах самая частая пара — PostgreSQL + Python. Её доля — 32,13% в Москве и около 36,2% в двух других регионах. Самая частая тройка в Москве и Санкт-Петербурге — Docker + PostgreSQL + Python. В Екатеринбурге лидирует Git + PostgreSQL + Python, но её медиана рассчитана только по пяти зарплатам.

Для тройки Docker + PostgreSQL + Python медианы Москвы и Санкт-Петербурга равны 225 000 и 135 000 руб. по 52 и 17 зарплатам. Одинаковый стек не означает одинаковую медиану в разных регионах.

7. Вывод

Для варианта № 2 обработаны 3 892 вакансии Python-разработчиков. MapReduce из S3 и независимый SQL-расчёт DuckDB полностью совпали. Повтор ошибочной задачи не изменил результат. DuckDB оказался быстрее по медиане времени, MapReduce потребовал меньше памяти по измеренной сумме RSS.

Зарплатные различия зависят от сочетания навыков, региона и состава вакансий. Для всех трёх регионов выделяется пара PostgreSQL + Python. Наибольшая общая медиана среди выбранных регионов получена для Москвы.